Guía3. Informe final Proyecto APT Asignatura Capstone

1. Informe final Proyecto APT

El objetivo de este informe es que describas los aspectos más relevantes de tu Proyecto APT. Es importante que fundamentes las decisiones que tuviste que tomar a lo largo del proceso.

A continuación, encontrarás distintos campos que deberás completar con la información solicitada, los que dan cuenta del resumen de tu proyecto APT y sus principales resultados.

Nombre del proyecto	CascoVision — Sistema de detección automatizada del uso de Equipos de Protección Personal (EPP) mediante visión computacional y balizas de alerta zonales conectadas por IP.
Área (s) de desempeño(s)	El proyecto se vincula a dos áreas de desempeño del Plan de Estudio de Ingeniería en Informática: • **Desarrollo e implantación de soluciones de software:** motor de IA, agente Edge, API, base de datos, dashboard y baliza zonal, desplegados con Docker. • **Gestión de proyectos informáticos:** planificación y control con metodología en cascada, EDT, carta Gantt, matriz de riesgos y puertas de fase.
Competencias	El proyecto aborda las cuatro competencias del perfil de egreso: • Realizar pruebas de certificación de productos y procesos con buenas prácticas de la industria (1.1, 1.2 y 1.3). • Gestionar proyectos informáticos, ofreciendo alternativas para la toma de decisiones (2.1 y 2.2). • Construir modelos de datos escalables en el tiempo (3.1 y 3.2). • Desarrollar una solución de software sistematizando el desarrollo y el mantenimiento (4.1, 4.2 y 4.3).
Resumen	CascoVision es un sistema que detecta la ausencia de casco y chaleco reflectante en faenas de construcción con las cámaras IP que ya existen, y que activa una baliza zonal sonora y visual en el sector de la infracción, sin identificar

	a las personas. El equipo aplica una metodología tradicional en cascada con puertas de fase. Al cierre de la semana 11 de 18 están cerradas las fases de planificación, requerimientos (SRS v1.0) y diseño (D2 v1.0 y plan de pruebas D3), y la construcción va en un 45 %: el agente Edge, el modelo exportado a ONNX, la API de incidentes, las migraciones de la base de datos y la integración continua funcionan, y la cadena cámara → detección → baliza → API se demostró de punta a punta. El modelo cumple el recall exigido en las reglas de casco (0,947) y chaleco (0,887), pero no el mAP@0,5 de 0,90 por clase. Quedan la baliza física, el dashboard, las alertas remotas, Docker, la integración y las pruebas finales.
Abstract	CascoVision is a system that detects missing hard hats and high-visibility vests on construction sites using the IP cameras already installed, and triggers an audible and visual zone beacon in the area where the violation occurs, without identifying workers. The team follows a traditional waterfall methodology with phase gates. By the end of week 11 of 18, the planning, requirements (SRS v1.0) and design phases (D2 v1.0 and the D3 test plan) are closed, and construction is 45% complete: the Edge agent, the model exported to ONNX, the incident API, the database migrations and continuous integration are working, and the camera → detection → beacon → API chain has been demonstrated end to end. The model meets the required recall for the hard hat (0.947) and vest (0.887) rules, but not the per-class mAP@0.5 target of 0.90. The physical beacon, the dashboard, remote alerts, Docker, integration and final testing are still pending.

Contenidos del informe final	
1. Relevancia del proyecto APT	**Problema.** La construcción representa cerca del 7 % del PIB de Chile, emplea a más de 600.000 personas y tiene una de las tasas de accidentabilidad más altas del país. Una parte de esos accidentes tiene un origen evitable: no usar casco o chaleco reflectante. La supervisión depende de prevencionistas de riesgos que no pueden cubrir todos los frentes de una faena al mismo tiempo. **Relevancia para la profesión.** Es un problema operativo que se resuelve con las herramientas de la disciplina: visión computacional, procesamiento en el borde (Edge), integración de dispositivos por red, modelos de datos y despliegue con contenedores. La solución debe además respetar la privacidad (Ley N° 19.628), lo que obliga a decisiones de arquitectura: procesamiento local y difuminado facial. **Dónde se ubica.** Faenas de construcción en Chile que ya cuentan con cámaras IP de vigilancia, a las que el sistema se conecta sin reemplazar hardware. **A quiénes afecta.** Directamente a los trabajadores de faena; en lo operativo, a prevencionistas de riesgos y jefes de terreno; en lo económico, a las empresas constructoras, expuestas a multas de 1 a 150 UTM por infracción, y a las mutualidades. **Innovación.** Hoy la supervisión del EPP es una inspección visual del prevencionista, y las cámaras de la faena solo graban. CascoVision se diferencia en cuatro puntos: • Avisa en el lugar: la alerta suena y se ve en la baliza de la misma zona de la infracción, no solo en la pantalla de un supervisor. • Procesa en el borde: el video no sale de la faena; al servidor solo llega el incidente con su evidencia.

	• Privacidad por diseño: los rostros se difuminan antes de guardar la evidencia y el sistema no identifica a las personas. • Usa la infraestructura existente: se conecta por RTSP a las cámaras IP que ya hay, sin hardware nuevo salvo la baliza. **Aporte de valor.** Supervisión continua de todas las cámaras de la faena, alerta inmediata en la zona y registro de cada incidente con evidencia, útil para fiscalizaciones y auditorías ISO 45001. En la demostración de la semana 11, con una cámara IP simulada, la baliza recibió la alerta entre 7 y 37 ms después de la detección y el incidente quedó registrado en la API con su evidencia difuminada. *[Por completar al cierre, S15] Agregar el aporte de valor medido en las pruebas finales (latencia de punta a punta con la baliza real y tasa de detección en la demostración).*
2. Objetivos	**Objetivo general.** Desarrollar e implementar CascoVision, un sistema de detección automatizada del uso de EPP basado en visión computacional y balizas de alerta zonales conectadas por IP, que permita a las empresas constructoras prevenir accidentes laborales, reducir su exposición a multas y respaldar el cumplimiento de la norma ISO 45001. **Objetivos específicos.** 1. Entrenar y validar un modelo de detección de objetos de la familia YOLO (YOLO26) capaz de identificar casco y chaleco reflectante con mAP@0,5 ≥ 0,90 y recall ≥ 0,85 en las reglas de casco y chaleco. 2. Desarrollar el agente Edge que se integra con cámaras IP estándar mediante ONVIF/RTSP, sin reemplazar el hardware del cliente. 3. Implementar el subsistema de alerta física: comunicación por red hacia la IP de la baliza zonal, que emite una alerta sonora y visual ante cada infracción detectada en su zona. 4. Construir el dashboard web de monitoreo, gestión de incidentes y reportería con evidencia fotográfica. 5. Integrar el canal de notificaciones remotas (SMS y correo electrónico) hacia el prevencionista de riesgos. 6. Diseñar y aplicar un plan de pruebas de validación del producto y del proceso, con criterios de aceptación medibles. 7. Documentar la arquitectura, el modelo de datos y el despliegue con Docker (manual técnico) para el traspaso de la solución. **Estado al 25-10.** El objetivo 2 está cumplido por RTSP (agente Edge terminado y probado con una cámara IP simulada; falta probarlo con una cámara real). El 1 se cumple en recall por regla, pero no en mAP@0,5. El 6 tiene el plan aprobado y falta su ejecución. Los objetivos 3, 4, 5 y 7 están en construcción.
3. Metodología	**Metodología: tradicional en cascada con puertas de fase y solapamiento controlado.** Se adoptó en la semana 7, en reemplazo del desarrollo iterativo incremental declarado en la Fase 1, a partir de la observación del docente. **Fases y procedimientos.** (1) Inicio y planificación, S1–S4: definición, EDT y Gantt. (2) Análisis de requerimientos, S5–S11: SRS (D1). (3) Diseño, S8–S12: arquitectura,

	modelo ER, UML, protocolo de la baliza, diseño Docker (D2) y plan de pruebas (D3). (4) Construcción, S9–S15: modelo, base de datos, API, agente Edge, firmware, dashboard, alertas y contenedores. (5) Integración y pruebas, S13–S15: evidencias de D3. (6) Implantación y cierre, S11–S15: despliegue con Docker Compose y manual técnico (D4). (7) Presentación, S16–S17. Cada fase cierra con un entregable aprobado que habilita la siguiente; una tarea puede adelantarse solo si no depende del entregable pendiente. **Control del proyecto.** Una planilla de seguimiento con avance ponderado por horas, curva S planificada contra real, matriz de riesgos y control de equidad de commits, revisada cada semana. Desde la semana 11, cada integrante trabaja en su rama (dev-iahn, dev-joaco, dev-fernando), integra en qa por pull request revisado por QA y solo qa pasa a main, que está protegida. Un flujo de GitHub Actions ejecuta las pruebas en cada pull request. **Pertinencia.** Los requerimientos eran estables, porque el producto ya había sido evaluado técnica y económicamente; los hitos del curso son fijos; la compra del hardware obligaba a cerrar el protocolo antes de construir; y el plan del equipo ya era secuencial. La cascada hace explícitas las dependencias y deja un entregable verificable por fase. Las puertas de fase se registraron: el SRS y el D2 se aprobaron el 23-10 y cerraron la fase de diseño, lo que respalda las competencias de gestión (2.1 y 2.2).
4. Desarrollo	**Etapas desarrolladas (corte al 25-10-2026, semana 11).** Avance real ponderado del 49,9 %, contra 52,9 % planificado. • Planificación cerrada (nota grupal de la Fase 1: 6,8). • Requerimientos cerrados: SRS v1.0 aprobado el 23-10, con requisitos funcionales, no funcionales y casos de uso. • Diseño cerrado: D2 v1.0 con arquitectura y despliegue, modelo ER y diccionario de datos, diagramas UML de clases, secuencia y componentes, protocolo de la baliza, mockups del dashboard y diseño Docker; plan de pruebas D3 con 44 casos. • Modelo: YOLO26s reentrenado con un dataset de 5 clases propias del producto (casco, sin casco, sin chaleco, persona y chaleco; 5.033 imágenes de entrenamiento) y exportado a ONNX. En CPU, el ciclo completo del motor tarda cerca de 88 ms por cuadro, holgado para 1 FPS por cámara. • Agente Edge terminado: captura RTSP a 1 FPS con reconexión, zona de riesgo por polígono, difuminado facial con YuNet, supresión de alertas repetidas, cliente de la baliza con reintento y heartbeat, y reporte asíncrono a la API con cola en disco para no perder incidentes si la API cae. • API FastAPI de incidentes con ciclo de inicio, ping y fin por episodio, y configuración del agente; migraciones PostgreSQL 001 a 005 con índices por zona y fecha y datos de prueba (las dos últimas en revisión). • Receptor simulado de la baliza y demostración de punta a punta con cámara IP simulada, modelo, baliza y API. • Integración continua con GitHub Actions, que ejecuta las 157 pruebas automatizadas en cada pull request a qa y main. *[Por completar al cierre, S15] Describir el firmware de la baliza StickS3, el dashboard React, las alertas por SMS y correo, Docker Compose, la integración en qa y los resultados de las pruebas.* **Facilitadores.** Proyecto ya evaluado en Evaluación de Proyectos; dataset público etiquetado; decisiones de diseño tomadas antes de construir; flujo de ramas con

	revisión por pull request y pruebas automáticas que detectan regresiones antes de llegar a main. **Dificultades.** Carga académica paralela; el mAP@0,5 por clase del modelo quedó entre 0,836 y 0,953, bajo el 0,90 en casco, sin casco y sin chaleco; el reentrenamiento necesita una GPU y el dataset completo, que solo están en un equipo; trabajo duplicado (dos implementaciones de las zonas de riesgo que, al integrarse, rompieron el agente); pull requests fusionados sin revisión de QA o aprobados por su propio autor; y atraso en el firmware de la baliza, el dashboard y Docker. **Ajustes realizados.** De incremental a cascada; de alerta vibratoria a baliza zonal sonora y visual; el agente Edge como emisor de la alerta; Docker en el alcance; de YOLOv8 a YOLO26 y de 11 a 5 clases, con clases negativas explícitas (sin casco, sin chaleco), lo que subió el recall de la regla de casco de 0,643 a 0,947; el RNF-05 se mide por regla de negocio y el mAP por clase se reporta como desviación; modelo en ONNX para servidores sin GPU; cámara IP simulada mientras no hay una real; nuevos roles y flujo de ramas desde la semana 11; numeración de migraciones acordada en el D2; y reglas de revisión de pull requests propuestas en la reunión de la semana 11. *[Por completar al cierre, S15] Agregar los ajustes posteriores a la semana 11 y cómo se resolvieron las dificultades abiertas (mAP, atrasos y proceso de revisión).*
5. Evidencias	**Evidencias en el repositorio oficial (rama main, salvo que se indique):** • Documentos: SRS v1.0 (D1), D2 v1.0 (diseño y arquitectura), plan de pruebas D3, carta Gantt y guía 2.4 (Fase 2/Evidencias Grupales/). • Diseño técnico: modelo de datos, protocolo de la baliza, diseño Docker, contrato de la API, mockups y siete diagramas (arquitectura, despliegue, casos de uso, clases, secuencia, componentes y ER) en docs/. • Programación: API FastAPI (src/api/), agente Edge (src/edge/) y motor de detección con difuminado (Yolo/modelamiento/). • Modelo: detector YOLO26s en .pt y .onnx con sus métricas por clase y por regla de negocio (models/README.md). • Almacenamiento de datos: migraciones PostgreSQL 001 a 003 en db/migrations/, y 004, 005 y datos de prueba en el pull request #24. • Pruebas: 157 pruebas automatizadas en tests/ y el flujo de integración continua en .github/workflows/tests.yml. • Gestión: historial de commits por integrante y pull requests revisados en GitHub, y planilla de seguimiento con curva S y matriz de riesgos. *[Por completar al cierre, S15] Agregar el D3 con resultados (mAP, recall, latencia menor a 5 s, seguridad y rendimiento), el D4, el docker-compose, capturas del dashboard y el video de la demostración con la baliza real.*
6. Intereses y proyecciones profesionales	**Iahn Vera.** Al iniciar el Capstone planteé tres intereses: desarrollo backend y de datos, visión computacional aplicada y gestión de proyectos informáticos. El proyecto confirmó el tercero con más fuerza de la que esperaba: el aporte que más impactó al equipo fue ordenar decisiones (metodología, tipo de alerta, emisor de la alerta) y sostener la coherencia entre requerimientos, plan y código. En la primera mitad del semestre mi aporte de código quedó por debajo del de mis compañeros; en la semana 11 lo equilibré construyendo el agente Edge y la exportación del modelo, y confirmé que me interesa tanto la integración de sistemas como la gestión.

	Proyección. Quiero profundizar en el desarrollo backend y de integración de sistemas (API, bases de datos, procesamiento en el borde) y combinarlo con la gestión técnica de proyectos, idealmente en un rol de desarrollador que participe en las decisiones de arquitectura. Administro una empresa de transporte, así que la seguridad operacional y la trazabilidad de incidentes son problemas que conozco desde el lado del usuario; CascoVision podría continuar como producto después de la titulación. *[Por completar al cierre, S15] Revisar esta reflexión al cierre del proyecto (¿siguen siendo los mismos intereses?).* **Joaquín Armijo · Benjamín Cáceres · Fernando Muñoz:** *[Pendiente: cada integrante redacta su propia reflexión sobre intereses y proyección profesional (criterio individual de la rúbrica final).]*